SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

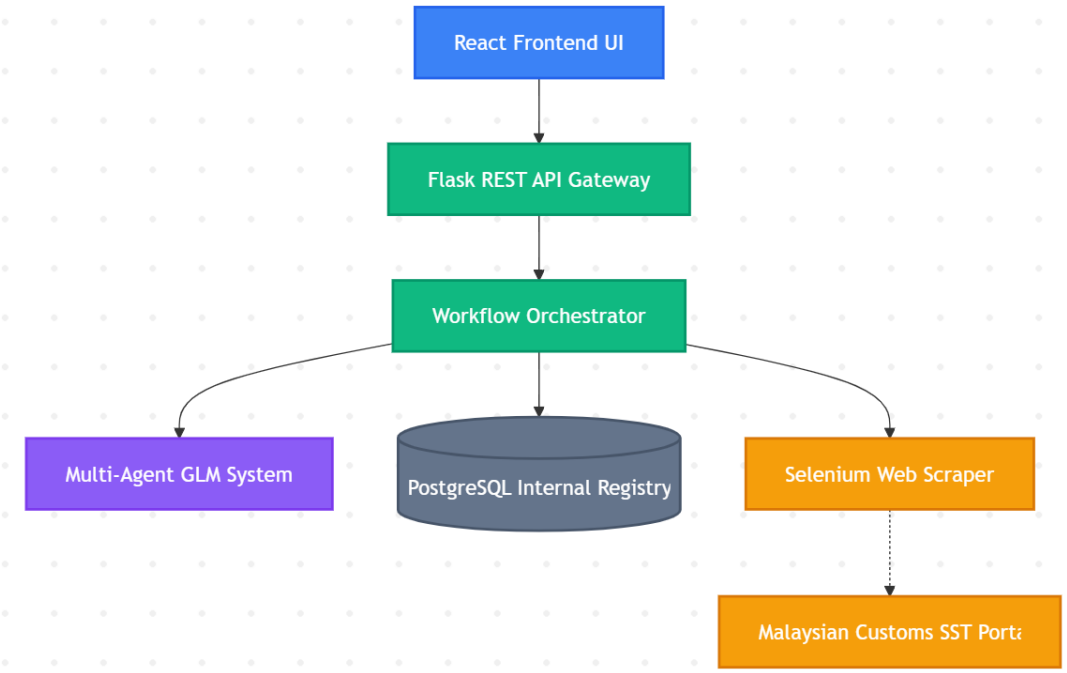
1. Introduction:

The EyeVen system introduces a strategic, AI-powered solution to automate the vendor onboarding and risk intelligence process. By replacing manual document reviews and fragmented government portal checks with an autonomous pipeline, EyeVen ensures complete compliance, prevents fraud, and significantly reduces the processing time for procurement officers.

1.1. Purpose:

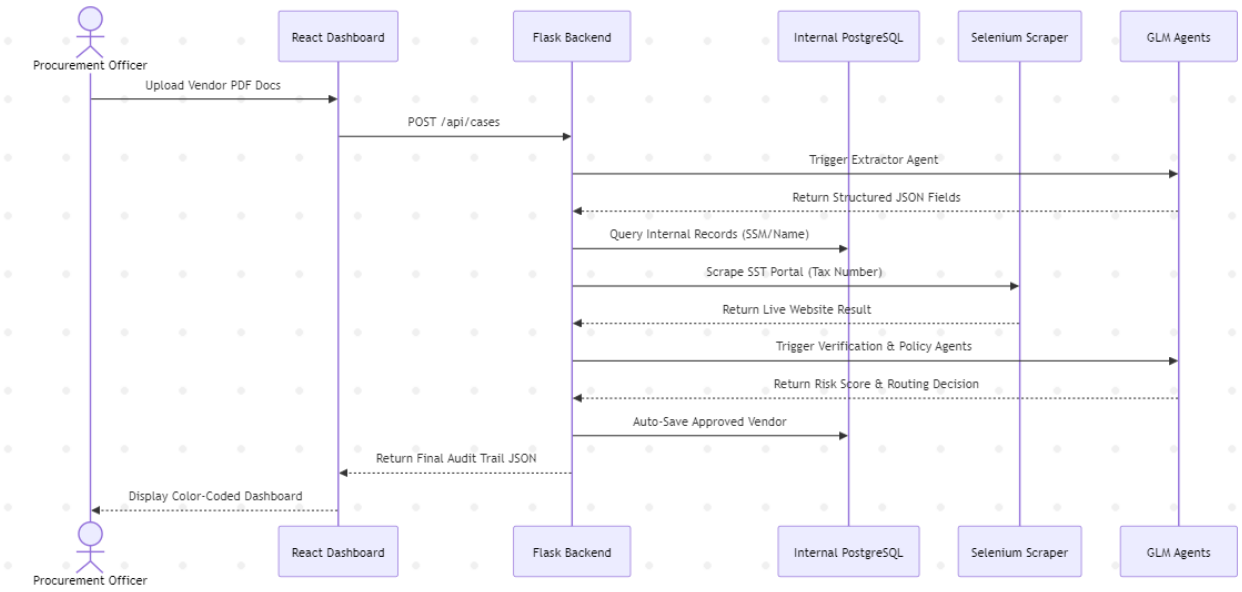
This system analysis document highlights the technical scope, multi-agent LLM architecture, and design-related decisions behind the development of EyeVen. It serves as a comprehensive guide to understanding how document extraction, internal registry lookups, and live external web scraping integrate into a single Flask-based state machine.
So, the key element of system that has been covered are as followed:

1. ***Architecture**:*



2. ***Data Flows:***
   The data flow begins when a procurement officer uploads a vendor's PDF documents. The Flask backend extracts the raw text and passes it to the Extractor GLM to generate structured JSON. This structured data is then cross-referenced by the backend against the internal PostgreSQL database and the live Malaysian Customs portal via Selenium. The aggregated facts are fed back into the Policy and Routing GLMs to generate a final risk decision, which is served back to the React dashboard.

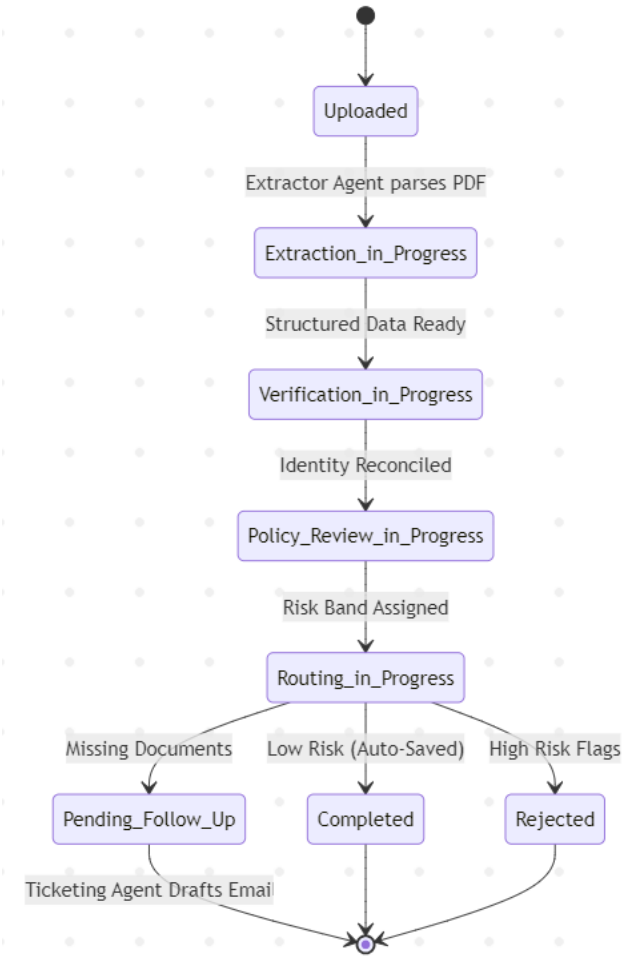
© UMHackathon 2026



3. ***Model Process:***

   EyeVen operates on a strict "State Machine" model process. A case must successfully pass through a sequential pipeline of highly specialized AI agents. If a critical failure or missing document is detected at any state, the orchestrator immediately branches the workflow to a "Rejected" or "Pending Follow-Up" state, triggering the Ticketing agent to draft an Outlook email.

© UMHackathon 2026



1.2. Background:

Procurement departments currently handle vendor onboarding via highly manual processes. Officers must read dense PDF submissions, manually search the vendor's SSM or Tax identifiers on slow government portals, and painstakingly compare the form data against supporting documents. This process takes days, is highly susceptible to human error, and creates severe compliance bottlenecks.

1. *Previous Version:*
   Prior to EyeVen, vendor onboarding was managed through fragmented email chains, physical document folders, and basic static web forms. These legacy systems lacked autonomous intelligence, requiring a human in the loop to validate every single data point and manually draft follow-up requests for incomplete applications.

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Procurement Officer	Main operational user who uploads vendor submissions, reviews structured vendor cases, checks follow-up drafts, uploads corrected documents, and monitors case progress.	A fast and structured onboarding workflow that reduces manual document reading, improves visibility of missing items, and supports timely vendor follow-up.
Compliance Officer	Reviews flagged or escalated vendor cases from a regulatory and policy perspective, especially when documents are incomplete, suspicious, or inconsistent.	Clear visibility into compliance evidence, mismatched fields, missing certifications, and risk-related findings so exception cases can be assessed efficiently.
Senior Management Approver	Handles high-risk or escalated vendor cases that require higher-level approval or rejection decisions.	A concise, decision-oriented case summary with verification outcomes, risk indicators, and recommended next actions instead of raw documents.
Internal Procurement / Operations Team	Oversees workflow progress, case queues, ticket history, and audit traceability across the onboarding process.	Better monitoring of vendor case status, more standardized workflows, and clear audit visibility for operational coordination.
Vendors	External party who submits onboarding documents and may later provide corrected or missing files when requested.	Clear requests for missing documents, faster onboarding decisions, and a better follow-up experience.

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*[e.g. Mobile App (iOS & Android)]*
Architecture	*[e.g. Cloud-Server with Microservices]*

[e.g. YFood is primarily structured as a client-server-based system in which there are 3 types of clients. 1. Customer Mobile Application, Rider Mobile, AND Vendors Web-based Dashboard. All these clients communicate by Backend API layer. So, three independent microservices: Order Service, Rider Assignment Service and Notification Service. All services are deployed on Amazon Web Services, and all share a PostgreSQL database that is hosted also in AWS RDS.]

2.1.2 LLM as Service Layer

The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic "AI Module". The following Dependency diagram includes:

2.1.3 Dependency Diagram

a. *How the prompts are being constructed and sent to the GLM API*
b. *What goes into the context window?*
c. *How is your system receiving, parsing the responses and passing to the next system component?*
d. *Where the limitation of token are enforced or input are chunked before it actually reaches to GLm*
e. *Also, do outline all the major API calls between the system components [e.g. Customer Mobile App calls "Amazon API Gateway" for Order Service]*
f. *Do show how frontend, backend, services, database and external API interacts with each other [e.g. Order service communicates with Amazon RDS to ensure persistence of order data]*
g. *Must highlights the distributed components if the system is planning to be deployed across multiple services*

2.2.2. Sequence Diagram

a. *Here, do walk through one user interaction flow or customer journey with the stakeholders [e.g. Place Order Flow]*
b. *Show the interactions step by step.*
c. *Do demonstration of each key system features does operate from start to finish*

© UMHackathon 2026

2.2. Technological Stack

2.2.1. Frontend

The frontend of EyeVen is developed using **React** with **TypeScript** to build a structured, maintainable, and interactive web-based dashboard. TypeScript is used to improve code reliability and reduce errors when handling structured workflow data such as vendor cases, extracted fields, verification results, routing outcomes, and ticket records. **Material UI (MUI)** is used as the main UI component library to create a clean and consistent dashboard interface for procurement officers, compliance officers, and senior approvers. The frontend supports key interactions such as document upload, structured vendor case viewing, escalated case review, ticket handling, and audit trail monitoring.

2.2.2. Backend

The backend of EyeVen is developed using **Python** with **Flask** to provide RESTful API services for vendor submission intake, document processing, vendor verification, workflow routing, ticket handling, and reprocessing of corrected submissions. Flask is selected because it is lightweight, flexible, and suitable for rapid prototype development within the hackathon timeline. The backend also coordinates the main workflow logic across the system, including file handling, OCR processing, structured extraction, policy evaluation, and routing decisions.

2.2.3. Charts / Dashboard Visualisation

EyeVen uses **Chart.js** to provide dashboard visualisations that help users understand workflow status and decision outcomes more clearly. These charts may be used to display metrics such as number of approved vendors, escalated cases, rejected submissions, missing-data tickets, and case progress distribution. This supports quicker operational monitoring and better decision visibility for internal teams.

2.2.4. OCR and Document Processing

EyeVen uses **PaddleOCR** as the OCR engine to extract text from scanned or image-based vendor documents, especially when uploaded PDFs are not fully text-readable. PaddleOCR supports the system in converting visual document content into machine-readable text before it enters the AI extraction and reasoning pipeline. This improves the system's ability to handle real-world vendor

© UMHackathon 2026

submissions such as scanned certificates, image-heavy compliance forms, and low-structure supporting documents.

2.2.5. Database

EyeVen uses **MySQL** as the main relational database for storing vendor case records, uploaded file metadata, extracted information, verification results, routing decisions, ticket history, and audit trail events. MySQL is chosen because it is suitable for structured relational data, supports reliable query processing, and is familiar to the development team. This makes it appropriate for the workflow-driven and data-linked design of the system.

2.2.6. Development Environment

The development of EyeVen is carried out in a **local-first environment** using **localhost** and **Visual Studio Code (VSCode)**. Localhost is used during implementation and testing to allow rapid iteration of the frontend, backend, database, and OCR pipeline without depending on paid infrastructure. VSCode is used as the primary development environment for coding, debugging, extension support, and project organisation.

2.2.7. Version Control and Collaboration

GitHub is used for version control, code repository management, and collaboration between team members. It allows both developers to work in parallel across different modules while maintaining a shared codebase, tracked commit history, and controlled integration of changes. This also supports cleaner documentation of implementation progress and repository management, which is relevant to the judging focus on version control and code structure.

2.2.8. Deployment

Since cloud services are not provided by the hackathon, EyeVen is designed to be developed and tested locally first, with lightweight deployment choices where feasible. The frontend is planned to be deployed on **Vercel** for demonstration purposes, while the backend may continue running locally during prototype presentation if full online deployment is not practical within the given

timeframe. This approach is realistic for the preliminary round and aligns with the event constraint that teams must manage their own tooling and hosting decisions.

2.2.9. Containerization

EyeVen uses **Docker** to standardize the development and runtime environment across team members' machines. Docker helps package the frontend, backend, database, and supporting services into isolated containers so that the application can run consistently regardless of local machine differences. This reduces environment setup issues, improves collaboration, and makes integration and testing more manageable.

2.2.10. LLM / Reasoning Engine

EyeVen uses **Z AI GLM** as the core large language model and reasoning engine of the system. Z AI GLM is responsible for extracting structured vendor information from unstructured inputs, identifying missing or ambiguous data, supporting agentic workflow decisions, and generating structured outputs used in verification, routing, and follow-up handling. This is also aligned with the hackathon requirement that Z AI must be the central reasoning model used in the solution.

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)

a. Illustrate the data movement across the entire system that includes external entities (users), processes, external service and databases

b. Show how proper laws are met (e.g. Black Hole)

2.3.2. **Normalized Database Schema**

a. Show the logical data model (ERD) with Database designed to 3NF form to ensure data integrity and relationships. [e.g. order items is a junction table that is resolving the M: N relationship between orders and menu items.

3. Functional Requirements & Scope

3.1. Minimum Viable Product:

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026. ***[e.g. in Table]***

#	Features	Description
1	Vendor Submission Intake & Structured Case Creation	Procurement officers upload vendor files such as company profiles, compliance certificates, or spreadsheets in either PDF, excel spreadsheet or bundle ZIP format. The system validates the files, parses the contents, and converts them into a structured vendor case.
2	AI-Based Vendor Information Extraction	Z AI GLM extracts key fields such as vendor identity, registration number, certificate information, expiry dates, and missing-field indicators from unstructured documents and outputs them in structured form.
3	Layered Vendor Verification & Policy Evaluation	The Verification AI agent in EyeVen verifies vendor legitimacy through an internal trusted vendor registry, external verification connectors, and document consistency checks. then evaluates the case against procurement and compliance rules
4	Intelligent Routing & Exception Handling	Based on verification and policy results evaluated by Policy Evaluation Agent, the system then uses Intelligent Routing Agent to route the case to approval, escalation, rejection, or missing-data handling. This includes support for escalated review by compliance officers or senior management.
5	Missing Data Ticketing & Reprocessing	When required information is missing or unclear, the system creates a ticket, prepares a follow-up draft email, and supports reprocessing when corrected files are later uploaded.

3.2. Non-Functional Requirements (NFRs)

© UMHackathon 2026

Quality	Requirements	Implementation
Scalability	The system should handle multiple vendor cases and repeated document-processing tasks without major redesigning to the architecture.	Use modular service separation between upload/parsing, AI extraction, verification, and routing logic. Keep workflow states and case records normalized in the database.
Reliability	Vendor cases should not proceed to final approval if extracted outputs are malformed, missing required fields, or fail verification.	Validate AI outputs against structured schema, enforce required-field checks, and fallback to ticket/manual review states when confidence is low or verification fails.
Maintainability	The system should be easy to extend, debug, and test across modules.	Organize code by functional modules such as intake/extraction/verification and workflow/routing/ticketing. Use typed API contracts, clear service boundaries, and documented workflows.
Token Latency	AI-based extraction and reasoning should be completed within a usable response time during normal demo conditions.	Pre-parse files locally, chunk long documents, pass only relevant content to Z AI GLM, and apply one-time retries rather than repeated uncontrolled calls. Apply OCR to reduce token usage of Z AI GLM.
Cost Efficiency	AI usage should remain practical for a hackathon-scale prototype and future scaled deployment.	Applying OCR and document preprocessing to convert scanned files into machine-readable text and filter only relevant sections before sending them to Z AI GLM, thereby reducing

© UMHackathon 2026

		token usage and improving cost efficiency.
Traceability	Every important workflow decision should be explainable and reviewable.	Store extracted fields, verification results, routing outcomes, ticket history, and reprocessing events in an audit trail table or timeline view.
Security & Data Integrity	Uploaded vendor documents and structured outputs should not be lost, corrupted, or mixed across cases.	Use server-side validation, relational foreign-key mapping, and controlled update logic when reprocessing corrected submissions.

3.3. Out of Scope / Future Enhancements

Do identify the high-level features that the team planned for "Grand Vision" but not part of this UMHackathon 2026. [e.g. below]

a. *During delivery, the In-App chat features between the rider and customer*
b. *Promotional offers, discount code, vouchers*
c. *Scheduled Delivery (Order now, and wants delivery to be delivered on a specific time)*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. ***Grayscale Rollout & A/B Testing****: Do highlight team's strategy of deployment [e.g. new feature of user-preferred food menu are released to 5% of real users initially, monitoring and then gradually increased until 100% if no issues are detected. After it passes, A/B testing is conducted to determine which variants of two strategies perform better.]*

4.1.2. ***Emergency Rollback (ER) & Golden Release****: Do mentioned the strategies for Emergencies. [e.g. In terms of recovery & traffic management for YFood system, if there are anomalies, ER is triggered. The automated pipeline does deploy the Golden Version of the application.]*

© UMHackathon 2026

4.1.3. ***Priority Matrix:***

4.1.3.1. Clearly define the Priority Matrix and what are the trigger conditions and what actions will be taken for your project. [e.g. P1 Critical: If payment failure error is more than 1%, emergency rollback to Golden release.]

4.2. Monitoring:

4.2.1. Show the planning for your monitoring plan of the system and how it will trigger if any damage is detected of the health of the system.

4.3. Assumptions: *State key operational & environmental conditions that you assumed to be valid while development of system and deployment. [e.g.*

a. Users have stable internet connections on their devices during the placement of orders
b. Restaurants have their own devices to access the web-dashboard for monitoring of orders]

4.4. External Dependencies: *Do list all the external tools that are applied to your system to function. [e.g. below]*

Tools	*Purpose*	*Risks*
Z Ai GLM API	*It should be the core LLM reasoning engine - that will handle the prompt inference, multi-step reasoning and generating structured output*	*High- in case of GLM API rate limitation - core product will fail. So, implement a mitigation strategy [e.g. graceful degradation state from users]*
Google Maps API	*Tracking rider location and optimize the route*	*Medium level- if real time tracking is being disabled.*

© UMHackathon 2026

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline: *Show the timeline of development and justify accordingly [e.g. Day 1-2, Project Planning etc.]*

5.2. Team Members Role: *Highlight the role of team members [e.g. Person A – Role Backend Lead – Fast API Microservices & Schema Design]*

Team Member	Modules/Tasks	Responsibilities
Chuah Yew Ken	Intake, Extraction & Verification Module	Handles vendor submission intake, file parsing, document preprocessing using OCR, Z AI GLM integration for structured extraction, internal trusted vendor registry checks, external verification connector logic, and document consistency validation. Also contributes to related UI sections such as upload and structured-case display.
Chin Ker Rou	Workflow, Routing & Reprocessing Module	Handles policy evaluation, risk assessment, routing logic, escalated-case workflow, missing-data ticketing, follow-up draft handling, reprocessing logic, audit trail flow, and related dashboard views such as status monitoring and escalated review.
Both Members	Shared Responsibilities	Collaborate on database design, architecture decisions, API contracts, QA planning, documentation writing, demo preparation, bug fixing, and final submission packaging.

5.3. Recommendations

To ensure future scalability, performance, and reliability, the following enhancements are recommended:

1. *Caching Layer: Implement Redis Cache to temporarily store external web scraping results. If a vendor submits a correction within 24 hours, the system can bypass the slow Selenium scrape and instantly retrieve the cached SST verification status.*

2. *Enterprise API Gateways: Transition from headless Selenium scraping to official B2B Government API Gateways (e.g., direct SSM or LHDN API integration) to eliminate timeout errors and reduce latency.*

3. *Role-Based Access Control (RBAC): Implement strict JWT-based authentication so "High Risk" overrides can only be executed by Senior Procurement Managers, ensuring an immutable compliance chain of custody.*

© UMHackathon 2026

Email: umhackathon@um.edu.my